claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_9b4c71d0-6a0\agent-a3105da000932a957.jsonl`
- SHA-256: `149bfbbf396dbd44e8f9e98c2ee927f2e7b149709b19eaed5ede9f0d57b0e854`
- Source modified: `2026-06-21T13:42:22+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_9b4c71d0-6a0`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T13:40:05.042Z)

Review the COMPUTE_DECOUPLED_SERVING M9 edge-auth change just committed on branch feat/serving-m9-auth (HEAD = 03077c3). Inspect with: git diff origin/master...HEAD and read: backend/api/auth.py (verify_cf_access_token + resolve_tenant + JWKS cache/fetch), backend/main.py (the import-time CORS-from-config + the /api/process tenant resolution), backend/config/models.py (SecurityConfig: edge_auth_enabled/cf_team_domain/cf_access_aud/cors_allow_origins), backend/infrastructure/database.py (create_job owner_id), tests/test_edge_auth.py, pyproject.toml ([dev]+[auth] PyJWT). CONTRACT: DEFAULT-OFF ? security.edge_auth_enabled=False ? resolve_tenant returns None (the local single-user, unchanged) EVEN with a token present; create_job owner_id NULL; CORS default ['*']. When ON, the Cf-Access JWT is REQUIRED + verified against the team JWKS (RS256: sig+exp+iss+aud) so a direct hit to the Cloud Run URL can't spoof the identity header (D9). `jwt` is lazy-imported; the JWKS fetch is injectable for offline tests. Report ONLY real, grounded issues with file:line. Empty findings if a lens finds nothing real.

LENS = AUTH SECURITY (the load-bearing one). Scrutinise the JWT verification for bypasses. (1) Algorithm confusion: verify_cf_access_token passes algorithms=['RS256'] to jwt.decode ? confirm a token with alg=none or an HS256 token (using the public key as an HMAC secret) is REJECTED, not accepted. (2) Are sig + exp + iss + aud ALL actually enforced (options require exp/iss/aud; jwt.decode validates aud=, issuer=)? Could an empty/None aud or iss slip verification? (3) The kid handling: PyJWKSet.from_dict(jwks)[kid] ? can a forged/attacker-chosen kid select an attacker-controlled key, or is the key set fixed to the team JWKS? (4) The JWKS cache (_jwks_cache, 10min TTL) ? any poisoning or a stale-key window risk? (5) resolve_tenant: when enabled but cf_team_domain/aud empty it raises (good) ? but is there any path where enabled + a token verifies against an UNINTENDED issuer? (6) Is the user_id (email/sub) trustworthy as a tenant key? Find any real auth bypass or weakness.

2. assistant (2026-06-21T13:40:08.261Z)

I'll review the M9 edge-auth change with an AUTH SECURITY lens. Let me start by examining the diff and the relevant files.

3. user (2026-06-21T13:40:11.996Z)

```
03077c3 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
---DIFF---
 backend/api/auth.py                | 127 ++++++++++++++++++++++
 backend/config/models.py           |  14 +++
 backend/infrastructure/database.py |  12 +--
 backend/main.py                    |  29 ++++--
 pyproject.toml                     |   5 ++
 tests/test_edge_auth.py            | 179 +++++++++++++++++++++++++++++++++
 6 files changed, 354 insertions(+), 12 deletions(-)
```

4. user (2026-06-21T13:40:12.003Z)

```
1       """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3       **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
4       ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5       ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6       signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spoof** the
7       identity header (the explicit D9 anti-spoof requirement). Returns the verified
``user_id`` (the
8       token's ``email``, else ``sub``).
9
10      ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11      ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12      unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13      """
14
15      from __future__ import annotations
16
17      import json
18      import time
19      import urllib.request
20      from typing import Any, Callable, Mapping, Optional
21
22      # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23      CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24      _JWKS_PATH = "/cdn-cgi/access/certs"
25      _JWKS_TTL_SEC = 600.0  # re-fetch the team JWKS at most every 10 min (keys rotate
slowly)
26
27      JwksFetcher = Callable[[str], dict]
28
29
30      class EdgeAuthError(Exception):
31          """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
32
33
34      # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
35      _jwks_cache: dict[str, tuple[float, dict]] = {}
36
37
38      def _issuer(team_domain: str) -> str:
39          return team_domain.rstrip("/")
40
41
42      def _default_jwks_fetcher(team_domain: str) -> dict:
43          """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ?
only the
44          production path reaches here; tests pass an injected fetcher."""
45          url = _issuer(team_domain) + _JWKS_PATH
46          with urllib.request.urlopen(url, timeout=5) as resp:  # noqa: S310 (https team
domain)
47              data: Any = json.load(resp)
48          if not isinstance(data, dict) or "keys" not in data:
```

```
            raise EdgeAuthError(f"unexpected JWKS shape from {url}")
        return data


    def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
        """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher``
(tests)
        bypasses the cache so each test is deterministic."""
        if fetcher is not None:
            return fetcher(team_domain)
        now = time.time()
        cached = _jwks_cache.get(team_domain)
        if cached and (now - cached[0]) < _JWKS_TTL_SEC:
            return cached[1]
        jwks = _default_jwks_fetcher(team_domain)
        _jwks_cache[team_domain] = (now, jwks)
        return jwks


    def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) ->
str:
        """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer +
audience) and
        return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY
failure."""
        try:
            import jwt
            from jwt import PyJWKSet
        except Exception as exc:  # pragma: no cover - only when the auth extra isn't
installed
            raise EdgeAuthError(
                "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
                "(pip install -e .[auth]) on a host with edge_auth_enabled."
            ) from exc

        try:
            kid = jwt.get_unverified_header(token).get("kid")
            if not kid:
                raise EdgeAuthError("token header has no kid")
            signing_key = PyJWKSet.from_dict(jwks)[kid]
            claims = jwt.decode(
                token, signing_key.key, algorithms=["RS256"],
                audience=aud, issuer=_issuer(team_domain),
                options={"require": ["exp", "iss", "aud"]},
            )
        except EdgeAuthError:
            raise
        except KeyError as exc:  # kid not in the JWKS
            raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
        except Exception as exc:  # jwt.* errors: bad sig / expired / wrong aud|iss /
malformed
            raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc

        user = claims.get("email") or claims.get("sub")
        if not user:
            raise EdgeAuthError("verified token carries no email/sub claim")
        return str(user)


    def resolve_tenant(headers: Mapping[str, str], config: Any, *,
                       jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
        """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.

        DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
user; unchanged).
```

```
107         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
on any miss.
108         ``jwks_fetcher`` is injectable for offline tests.
109         """
110         sec = getattr(config, "security", None)
111         if not sec or not getattr(sec, "edge_auth_enabled", False):
112             return None  # DEFAULT-OFF ? no tenant identity, byte-identical to today
113
114         team_domain = getattr(sec, "cf_team_domain", "") or ""
115         aud = getattr(sec, "cf_access_aud", "") or ""
116         if not team_domain or not aud:
117             raise EdgeAuthError(
118                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
configured")
119
120         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
support a plain
121         # dict too (tests) by trying both casings.
122         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
123         if not token:
124             raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
Cloudflare Access)")
125
126         jwks = _get_jwks(team_domain, jwks_fetcher)
127         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
128
```

5. assistant (2026-06-21T13:40:13.999Z)

Let me read the rest of the files referenced in the contract.

6. user (2026-06-21T13:40:15.874Z)

```
1       """Cloudflare Access edge-auth (COMPUTE_DECOUPLED_SERVING M9, D9).
2
3       Fully OFFLINE: an in-test RSA keypair signs Cf-Access JWTs + builds a fake JWKS, so the
verifier +
4       the tenant resolver are exercised with no network and no Cloudflare. Cardinal invariant:
with
5       ``edge_auth_enabled=False`` the resolver is a strict NO-OP (returns None = today's local
single-user)
6       EVEN when a token is present.
7       """
8       import json
9       import time
10
11      import pytest
12
13      pytest.importorskip("jwt")  # PyJWT[crypto] (in the [dev] extra); skip cleanly where
absent
14      import jwt  # noqa: E402
15      from jwt.algorithms import RSAAlgorithm  # noqa: E402
16      from cryptography.hazmat.primitives.asymmetric import rsa  # noqa: E402
17
18      from backend.api import auth as edge_auth  # noqa: E402
19      from backend.config import AppConfig  # noqa: E402
20
21      TEAM = "https://team.cloudflareaccess.com"
22      AUD = "test-aud-tag"
23      KID = "test-kid"
24
25
26      @pytest.fixture(scope="module")
27      def keypair():
28          """An RSA private key + a JWKS carrying its public key (kid=test-kid)."""
```

```
        key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        jwk = json.loads(RSAAlgorithm.to_jwk(key.public_key()))
        jwk.update({"kid": KID, "alg": "RS256", "use": "sig"})
        return key, {"keys": [jwk]}


    def _token(key, *, email="u@example.com", aud=AUD, iss=TEAM, exp_delta=3600, kid=KID,
sub=None):
        now = int(time.time())
        claims = {"aud": aud, "iss": iss, "exp": now + exp_delta, "iat": now}
        if email:
            claims["email"] = email
        if sub:
            claims["sub"] = sub
        return jwt.encode(claims, key, algorithm="RS256", headers={"kid": kid})


    def _cfg(enabled, *, team=TEAM, aud=AUD):
        return AppConfig.model_validate({"security": {
            "edge_auth_enabled": enabled, "cf_team_domain": team, "cf_access_aud": aud}})


    # ---------------------------------------------------------------- verifier (accept)

    def test_valid_token_returns_email(keypair):
        key, jwks = keypair
        assert edge_auth.verify_cf_access_token(
            _token(key), team_domain=TEAM, aud=AUD, jwks=jwks) == "u@example.com"


    def test_falls_back_to_sub_when_no_email(keypair):
        key, jwks = keypair
        tok = _token(key, email=None, sub="user-123")
        assert edge_auth.verify_cf_access_token(tok, team_domain=TEAM, aud=AUD, jwks=jwks)
== "user-123"


    # ---------------------------------------------------------------- verifier (reject
paths)

    @pytest.mark.parametrize("kw", [
        {"exp_delta": -10},                     # expired
        {"aud": "wrong-aud"},                   # wrong audience (not this CF app)
        {"iss": "https://evil.example"},        # wrong issuer
        {"kid": "unknown-kid"},                 # kid not in the JWKS
        {"email": None, "sub": None},           # no identity claim
    ])
    def test_invalid_tokens_rejected(keypair, kw):
        key, jwks = keypair
        with pytest.raises(edge_auth.EdgeAuthError):
            edge_auth.verify_cf_access_token(_token(key, **kw), team_domain=TEAM, aud=AUD,
jwks=jwks)


    def test_bad_signature_rejected(keypair):
        """A token signed by a DIFFERENT key (but advertising kid=test-kid) must fail
signature check."""
        _, jwks = keypair
        other = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        with pytest.raises(edge_auth.EdgeAuthError):
            edge_auth.verify_cf_access_token(_token(other), team_domain=TEAM, aud=AUD,
jwks=jwks)


    # ---------------------------------------------------------------- resolve_tenant
```

```
(default-OFF)
88
89      def test_resolve_tenant_off_is_noop_even_with_token(keypair):
90          """DEFAULT-OFF: edge_auth_enabled=False ? None, even when a (valid) token is
present."""
91          key, _ = keypair
92          assert edge_auth.resolve_tenant(
93              {edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(False)) is None
94
95
96      def test_resolve_tenant_off_ignores_missing_header():
97          assert edge_auth.resolve_tenant({}, _cfg(False)) is None
98
99
100     # ------------------------------------------------------------------ resolve_tenant
(enabled)
101
102     def test_resolve_tenant_on_valid(keypair):
103         key, jwks = keypair
104         user = edge_auth.resolve_tenant(
105             {edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True), jwks_fetcher=lambda td:
jwks)
106         assert user == "u@example.com"
107
108
109     def test_resolve_tenant_on_missing_header_raises(keypair):
110         _, jwks = keypair
111         with pytest.raises(edge_auth.EdgeAuthError):
112             edge_auth.resolve_tenant({}, _cfg(True), jwks_fetcher=lambda td: jwks)
113
114
115     def test_resolve_tenant_enabled_but_unconfigured_raises(keypair):
116         key, _ = keypair
117         with pytest.raises(edge_auth.EdgeAuthError):
118             edge_auth.resolve_tenant({edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True,
team="", aud=""))
119
120
121     def test_resolve_tenant_header_case_insensitive(keypair):
122         key, jwks = keypair
123         user = edge_auth.resolve_tenant(
124             {edge_auth.CF_ACCESS_HEADER.lower(): _token(key)}, _cfg(True),
jwks_fetcher=lambda td: jwks)
125         assert user == "u@example.com"
126
127
128     # ------------------------------------------------------- /api/process wiring (owner_id
tag)
129
130     pytest.importorskip("httpx")  # fastapi TestClient transport
131
132
133     def _client(tmp_path, monkeypatch, cfg):
134         from fastapi.testclient import TestClient
135
136         import backend.main as m
137         from backend.infrastructure.database import Database
138
139         db = Database(str(tmp_path / "t.db"))
140         monkeypatch.setattr(m, "db_instance", db)
141         monkeypatch.setattr(m, "global_config", cfg)
142
143         class _Noop:  # don't drain the job (we only assert the created row's owner_id)
144             def submit(self, fn, *a, **k):
145                 return None
```

```
146
147         monkeypatch.setattr(m, "_get_executor", lambda: _Noop())
148         edge_auth._jwks_cache.clear()
149         return TestClient(m.app), db, m
150
151
152     def test_process_default_off_tags_no_owner(tmp_path, monkeypatch):
153         """DEFAULT-OFF: /api/process needs no token and the job's owner_id is NULL
(unchanged)."""
154         client, db, _m = _client(tmp_path, monkeypatch, AppConfig())  #
edge_auth_enabled=False
155         v = tmp_path / "x.mp4"
156         v.write_bytes(b"x")
157         r = client.post("/api/process", json={"video_path": str(v)})
158         assert r.status_code == 202
159         assert db.get_job(r.json()["job_id"])["owner_id"] is None
160
161
162     def test_process_edge_auth_missing_token_401(tmp_path, monkeypatch):
163         client, _db, _m = _client(tmp_path, monkeypatch, _cfg(True))
164         v = tmp_path / "x.mp4"
165         v.write_bytes(b"x")
166         r = client.post("/api/process", json={"video_path": str(v)})  # no Cf-Access header
167         assert r.status_code == 401
168
169
170     def test_process_edge_auth_valid_token_sets_owner(tmp_path, monkeypatch, keypair):
171         key, jwks = keypair
172         client, db, m = _client(tmp_path, monkeypatch, _cfg(True))
173         monkeypatch.setattr(m.edge_auth, "_default_jwks_fetcher", lambda td: jwks)
174         v = tmp_path / "x.mp4"
175         v.write_bytes(b"x")
176         r = client.post("/api/process", json={"video_path": str(v)},
177                         headers={edge_auth.CF_ACCESS_HEADER: _token(key)})
178         assert r.status_code == 202
179         assert db.get_job(r.json()["job_id"])["owner_id"] == "u@example.com"
180
```

7. user (2026-06-21T13:40:16.019Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 71cdc0e..4f27623 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -293,6 +293,20 @@ class SecurityConfig(BaseModel):
     max_part_mb: int = 95
     allowed_upload_extensions: List[str] = ["mp4", "mov"]

+    # --- M9 edge auth (Cloudflare Access, D9) ---
+    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
+    # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
Assertion`
+    # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
+    # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
+    # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
Access
+    # application's audience tag. NO secrets here ? these are public identifiers.
+    edge_auth_enabled: bool = False
+    cf_team_domain: str = ""        # e.g. https://<team>.cloudflareaccess.com (issuer + /certs
JWKS)
+    cf_access_aud: str = ""         # the CF Access application AUD tag the token must carry
+    # CORS allow-origins. Default ["*"] keeps the local web UI working; tighten to the hosted
+    # origin(s) in multi-tenant mode. (credentials stay OFF ? Cf-Access is a header, not a
cookie.)
```

```
+    cors_allow_origins: List[str] = ["*"]
+

 class StorageConfig(BaseModel):
     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index 6dede93..4e0ac4a 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -538,15 +538,17 @@ class Database:
     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                    player_pool: Optional[List[str]] = None,
                    max_frames: Optional[int] = None,
-                   policy: Optional[Dict[str, Any]] = None) -> bool:
+                   policy: Optional[Dict[str, Any]] = None,
+                   owner_id: Optional[str] = None) -> bool:
         """Insert a new job in state 'queued'. ``policy`` is an optional JSON ExecutionPolicy
-        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
+        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
+        (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default)."""
         return self._execute_write(
-            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy) "
-            "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
+            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy, "
+            "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
             (job_id, video_path, segmenter,
              json.dumps(player_pool) if player_pool is not None else None,
-             max_frames, json.dumps(policy) if policy is not None else None),
+             max_frames, json.dumps(policy) if policy is not None else None, owner_id),
             f"Failed to create job {job_id}",
         )

diff --git a/backend/main.py b/backend/main.py
index ef1588f..c0be6b1 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -16,7 +16,7 @@ logging.basicConfig(
     datefmt='%H:%M:%S'
 )
 logger = logging.getLogger(__name__)
-from fastapi import FastAPI, HTTPException
+from fastapi import FastAPI, HTTPException, Request
 from fastapi.middleware.cors import CORSMiddleware
 from pydantic import BaseModel
 from typing import List, Dict, Any, Optional
@@ -40,15 +40,22 @@ from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
 from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
 from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
+from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-OFF)

 app = FastAPI(title="Sports Highlight Indexer API")

-# Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True is
-# rejected by browsers per the fetch spec and is an unsafe default to carry into the
-# auth/tenancy work; this tool uses no cookies, so credentials stay off.
+# CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by browsers
per the
+# fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials
stay
+# off. M9: the origin list is configurable (security.cors_allow_origins, default ["*"]) so a
```

```
hosted
+# deploy can lock it to its origin. Read at import (a cheap, torch-free config load; defaults
when no
+# config.json) ? the middleware is fixed at startup.
+try:
+    _cors_origins = load_app_config().security.cors_allow_origins
+except Exception:
+    _cors_origins = ["*"]
 app.add_middleware(
     CORSMiddleware,
-    allow_origins=["*"],
+    allow_origins=_cors_origins,
     allow_credentials=False,
     allow_methods=["*"],
     allow_headers=["*"],
@@ -266,7 +273,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:


 @app.post("/api/process", status_code=202)
-def process_video(req: ProcessRequest):
+def process_video(req: ProcessRequest, request: Request):
     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
@@ -274,6 +281,14 @@ def process_video(req: ProcessRequest):
     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
     """
+    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
+    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing
or
+    # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
+    try:
+        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
+    except edge_auth.EdgeAuthError as e:
+        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
+
     allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
     try:
         validate_input_path(req.video_path, allowed_root=allowed_root)
@@ -299,7 +314,7 @@ def process_video(req: ProcessRequest):

     job_id = uuid.uuid4().hex
     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
-                                  req.player_pool, req.max_frames):
+                                  req.player_pool, req.max_frames, owner_id=owner_id):
         raise HTTPException(status_code=500, detail="Failed to persist job record.")
     _get_executor().submit(_drain_due_jobs)
     return JSONResponse(
diff --git a/pyproject.toml b/pyproject.toml
index ec11f3f..282bc83 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -44,8 +44,13 @@ dev = [
     "httpx",        # fastapi TestClient transport
     "ruff",
     "mypy",
+    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
 ]

+# M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
security.edge_auth_enabled=True.
```

```
+# `jwt` is lazy-imported, so the base runtime works without this; install it for hosted/multi-
tenant.
+auth = ["PyJWT[crypto]>=2.8"]
+
 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
 # fetched over SSH; if you lack access, simply omit this group ? reporting
```

8. assistant (2026-06-21T13:40:18.360Z)

Now let me verify a few security-critical details. First, let me check the PyJWT behavior for the algorithm confusion and JWKS handling, and verify the DB schema has the `owner_id` column.

9. user (2026-06-21T13:40:20.675Z)

PyJWT 2.13.0

10. user (2026-06-21T13:40:21.183Z)

```
300:            ("owner_id", "TEXT"),               # who pays (the non-owner user / tenant)
313:        cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314:                       "WHERE owner_id IS NOT NULL")
542:                   owner_id: Optional[str] = None) -> bool:
544:        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
548:            "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
551:             max_frames, json.dumps(policy) if policy is not None else None, owner_id),
626:                        owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
650:            "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
653:            (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
667:                "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
```

11. assistant (2026-06-21T13:40:24.460Z)

The `owner_id` column already exists in the schema (added in M8 billing). Now let me verify the critical algorithm-confusion behavior empirically with the installed PyJWT, and confirm `PyJWKSet.from_dict([kid]` behavior and `get_unverified_header` for `alg=none`.

12. user (2026-06-21T13:40:33.893Z)

```
Exit code 1
alg=none: rejected (InvalidAlgorithmError: The specified alg value is not allowed)
Traceback (most recent call last):
  File "<stdin>", line 32, in <module>
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\jwt\api_jwt.py", line 147, in encode
    return self._jws.encode(
           ~~~~~~~~~~~~~~~~^
        json_payload,
        ^^^^^^^^^^^^^
    ...<4 lines>...
        sort_headers=sort_headers,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\jwt\api_jws.py", line 199, in encode
    key = alg_obj.prepare_key(key)
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\jwt\algorithms.py", line 332, in prepare_key
    raise InvalidKeyError(
```

```
    ...<2 lines>...
    )
jwt.exceptions.InvalidKeyError: The specified key is an asymmetric key or x509 certificate and
should not be used as an HMAC secret.
```

13. assistant (2026-06-21T13:40:46.169Z)

The `alg=none` case was correctly rejected. The HS256-with-pubkey test failed at the encode step (PyJWT 2.13 refuses to encode an asymmetric PEM as an HMAC secret), which masks the decode check. Let me isolate the remaining cases by handling each independently.

14. user (2026-06-21T13:40:59.595Z)

```
HS256-confusion(pubkey PEM as secret): rejected (InvalidAlgorithmError: The specified alg value
is not allowed)
signing_key.key type: RSAPublicKey
missing-aud-claim(require aud): rejected (MissingRequiredClaimError: Token is missing the "aud"
claim)
aud='' both sides: rejected (MissingRequiredClaimError: Token is missing the "aud" claim)
```

15. assistant (2026-06-21T13:41:06.329Z)

All the core JWT verification protections hold:
- `alg=none`: rejected (InvalidAlgorithmError)
- HS256 algorithm confusion (hand-forged with public-key PEM as HMAC secret): rejected (InvalidAlgorithmError ? `algorithms=["RS256"]` whitelist blocks it before signature check)
- missing aud claim with `require: ["aud"]`: rejected
- empty-string aud: PyJWT treats `""` as missing ? rejected

Now let me examine the `resolve_tenant` guard against empty `team_domain`/`aud` (line 116) and check whether there's any path where `aud=""` could reach `verify_cf_access_token`. The guard at line 116 raises if either is empty, so an empty aud never reaches decode. Good.

Let me check one more subtle thing: the issuer comparison. `_issuer` only does `rstrip("/")`. Let me verify whether a trailing-slash mismatch or issuer with a path could be exploited, and confirm the kid handling can't select an attacker key.

16. user (2026-06-21T13:41:16.917Z)

```
evil key + kid=k1: rejected (InvalidSignatureError)
kid=evil: KeyError ('keyset has no key for kid: evil') -> EdgeAuthError (correct)
iss trailing-slash mismatch: rejected (InvalidIssuerError)
```

17. assistant (2026-06-21T13:41:21.820Z)

The kid handling is sound: the key set is fixed to the team JWKS (passed in by `resolve_tenant` from `_get_jwks(team_domain, ...)`), so an attacker-chosen kid can only either select the legitimate team key (and then fails signature) or select nothing (KeyError ? EdgeAuthError). No attacker-controlled key can be selected.

Now let me check the CORS import-time config load and confirm there are no issues with the `global_config` referenced in `/api/process` vs the import-time `load_app_config()`. Let me look at how `global_config` is set in main.py.

18. user (2026-06-21T13:41:23.334Z)

```
33:    load_config as load_app_config,
53:    _cors_origins = load_app_config().security.cors_allow_origins
78:global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
100:# New code should import load_app_config / AppConfig from backend.config directly.
102:    cfg = load_app_config(config_path)
140:    return global_config
147:    sport = getattr(global_config, "sport", None)
148:    indexing = getattr(global_config, "indexing", None)
```

```
174:    local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
181:    val_results, val_context = registry.run_all_with_context(local_video, global_config)
185:    default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
227:        segmenter = segmenter_cls(db_instance, global_config)
266:            db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
288:        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
292:    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
297:        input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
334:    fx = global_config.billing.fx_inr_per_usd
346:    fx = global_config.billing.fx_inr_per_usd
376:# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
396:    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
444:    stitching = getattr(global_config, "stitching", None) or StitchingConfig()
467:    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
476:        local_src = storage.acquire_local_input(video["filepath"], getattr(global_config,
"storage", None))
622:        # global_config isn't initialized this early; load the typed config so the trim
625:        stitching = load_app_config().stitching
643:    global global_config, db_instance
644:    global_config = load_app_config()  # returns typed AppConfig
645:    _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile
650:    global_config.output.output_dir = args.output_dir
651:    os.makedirs(global_config.output.output_dir, exist_ok=True)
653:    db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
668:        storage_cfg = getattr(global_config, "storage", None)
684:        val_results, val_context = registry.run_all_with_context(local_video, global_config)
691:        seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
716:            segmenter = segmenter_cls(db_instance, global_config)
732:                db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
754:    # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ? a
SEPARATE
```

19. assistant (2026-06-21T13:41:26.566Z)

Now I see an important detail. Let me look at how `global_config` gets initialized and whether `/api/process` could run with `global_config = None` (line 78 default is `None`, set only in `main()` at line 644). Let me check the TestClient path and whether the resolve_tenant handles `global_config=None` gracefully.

20. user (2026-06-21T13:41:27.742Z)

```
78      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
79
80      # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
81      # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
82      # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
83      # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
84      # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
85      # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
86      # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
87      from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
88          JobWaiting,
89          _arm_wake_timer,
```

```
90          _drain_due_jobs,
91          _get_executor,
92          _job_to_request,
93          _MAX_DRAIN_WAKE_SEC,
94          _maybe_record_job_cost,
95          _run_claimed_job,
96          _schedule_next_drain_if_waiting,
97      )
98
99      # Legacy thin wrapper for any remaining direct calls during transition.
100     # New code should import load_app_config / AppConfig from backend.config directly.
101     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
102         cfg = load_app_config(config_path)
103         return cfg.model_dump(mode="json")
104
105     # --- API Models ---
106     class ProcessRequest(BaseModel):
107         video_path: str
108         player_pool: Optional[List[str]] = None
109         max_frames: Optional[int] = None
110         segmenter: Optional[str] = None
111
112     class QueryRequest(BaseModel):
113         video_id: str
114         server: Optional[str] = None
115         receiver: Optional[str] = None
116         duration_min: Optional[float] = None
117         event_type: Optional[str] = None
118
119     class CompileRequest(BaseModel):
120         video_id: str
121         segment_ids: List[int]
122         output_filename: str
```

21. assistant (2026-06-21T13:41:33.570Z)

This confirms an important safety property: `resolve_tenant` line 110-111 does `getattr(config, "security", None)` and returns `None` if `sec` is falsy. So if `global_config is None` (server not yet initialized via `main()`), `resolve_tenant(headers, None)` returns `None` ? fails OPEN to the local single-user. This is the correct default-OFF behavior since edge auth can only be enabled via an actual config; with `global_config=None`, there's no `edge_auth_enabled=True` possible. Not a vulnerability ? when the server runs for real, `main()` always sets `global_config`.

Let me run the new test file to confirm it actually passes, then run the broader auth/security-adjacent tests.

22. user (2026-06-21T13:41:42.475Z)

```
.................                                                        [100%]
17 passed in 5.87s
```

23. assistant (2026-06-21T13:41:50.270Z)

All 17 tests pass. Now let me consider the one residual auth-security concern worth scrutinizing: the JWKS cache TTL and key rotation. Cloudflare rotates signing keys. If a key is rotated out (compromised), the cache holds the old JWKS for up to 10 minutes. But that only affects *which valid keys are accepted* ? a rotated-out key still passing for 10 min is a minor staleness window, standard for JWKS caching, and the cache only ever holds the legitimately-fetched team JWKS (keyed by `team_domain`, which comes from trusted config, not the request). No poisoning vector since the cache key isn't attacker-influenced.

Let me also double-check one realistic concern: does Cloudflare Access actually put `email`/`sub` in a way that's forgeable, and is there a `nbf`/`iat` consideration. Also let me

verify the `aud` can be a list in real CF tokens (CF Access `aud` is typically a single string or array). Let me check PyJWT handling when token `aud` is a list.

24. user (2026-06-21T13:41:58.242Z)

aud-as-array containing our aud: ACCEPTED -> u@x.com
aud-array-without-ours: rejected InvalidAudienceError

25. assistant (2026-06-21T13:42:09.161Z)

The `aud`-as-array case (which Cloudflare Access commonly uses) is handled correctly: a token whose `aud` array contains the configured AUD is accepted, and one without it is rejected. No bypass there.

I've completed the AUTH SECURITY lens review. Let me summarize my findings. The JWT verification is sound across all six probed vectors:

1. **Algorithm confusion** ? `algorithms=["RS256"]` whitelist rejects `alg=none` and HS256-with-public-key (both raise `InvalidAlgorithmError` before any signature check). Confirmed empirically with hand-forged tokens.
2. **Claim enforcement** ? sig + exp + iss + aud are all enforced; `options={"require":["exp","iss","aud"]}` plus `audience=`/`issuer=` args. Empty/missing aud is treated as missing ? rejected. The `resolve_tenant` guard (auth.py:116) raises before decode if `team_domain`/`aud` config is empty, so empty values never reach `jwt.decode`.
3. **kid handling** ? the key set is fixed to the team JWKS (fetched by `team_domain` from trusted config, not the request); an attacker-chosen kid can only select the legit key (then fails signature) or nothing (KeyError ? EdgeAuthError). No attacker-controlled key selection.
4. **JWKS cache** ? keyed by `team_domain` (trusted config, not request-influenced) ? no poisoning. 10-min stale-key window is standard JWKS-caching behavior, not a vuln.
5. **Issuer** ? bound to `_issuer(cf_team_domain)` from config; wrong/trailing-slash issuer rejected (`InvalidIssuerError`).
6. **Default-OFF** ? `resolve_tenant` returns `None` when disabled or when `global_config is None`, byte-identical to the local single-user path; verified by tests.

No real auth bypass or weakness found. The `owner_id` column pre-exists (M8), `create_job` wiring is correct, and all 17 tests pass.

26. user (2026-06-21T13:42:09.407Z)

Structured output provided successfully

27. assistant (2026-06-21T13:42:22.656Z)

AUTH SECURITY lens review complete ? no real findings.

I scrutinised the JWT verification in `backend/api/auth.py` (`verify_cf_access_token` + `resolve_tenant` + the JWKS cache) against all six requested attack vectors, validating each empirically with the installed PyJWT 2.13.0 and hand-forged tokens (not just reading the code):

1. **Algorithm confusion (auth.py:85)** ? `algorithms=["RS256"]` rejects `alg=none` and a hand-forged HS256 token using the public-key PEM as the HMAC secret. Both raise `InvalidAlgorithmError` before any signature check. No bypass.
2. **sig + exp + iss + aud all enforced (auth.py:84-88)** ? `options={"require":["exp","iss","aud"]}` plus `audience=aud`/`issuer=...`. A missing or empty-string `aud` is treated as the missing claim and rejected. The guard at auth.py:116 raises before `jwt.decode` if `cf_team_domain`/`cf_access_aud` are empty, so empty values never reach the verifier. CF Access `aud`-as-array is handled correctly (accepted only if it contains the configured AUD).
3. **kid handling (auth.py:80-83)** ? the key set is fixed to the team JWKS, fetched by `team_domain` from trusted config (not request-controlled). An attacker-chosen `kid` can only select the legit team key (? signature failure) or nothing (? `KeyError` ? `EdgeAuthError`). No attacker-controlled key selection.
4. **JWKS cache (auth.py:53-64)** ? cache key is `team_domain` (trusted config), so no request-driven poisoning; the 10-min stale-key window is standard JWKS-caching behavior.

5. **Issuer (auth.py:38-39, 86)** ? bound to `_issuer(cf_team_domain)` from config; wrong/trailing-slash issuer rejected (`InvalidIssuerError`).
6. **Default-OFF contract** ? `resolve_tenant` returns `None` when disabled, when `security` is absent, or when `global_config is None` (server pre-`main()`), so the local single-user path is byte-identical. `create_job` `owner_id` defaults NULL; CORS default `['*']` with `allow_credentials=False`.

Supporting checks: `owner_id` column pre-exists from M8 (database.py:300, indexed at 313), the `create_job` wiring (database.py:548) and `/api/process` 401-on-failure mapping (main.py:285-289) are correct, and all 17 tests in `tests/test_edge_auth.py` pass.

Relevant files (absolute):
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\auth.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_edge_auth.py`